Module 6

Complex Multi-Agent & Protocol Testbenches (SV/UVM)

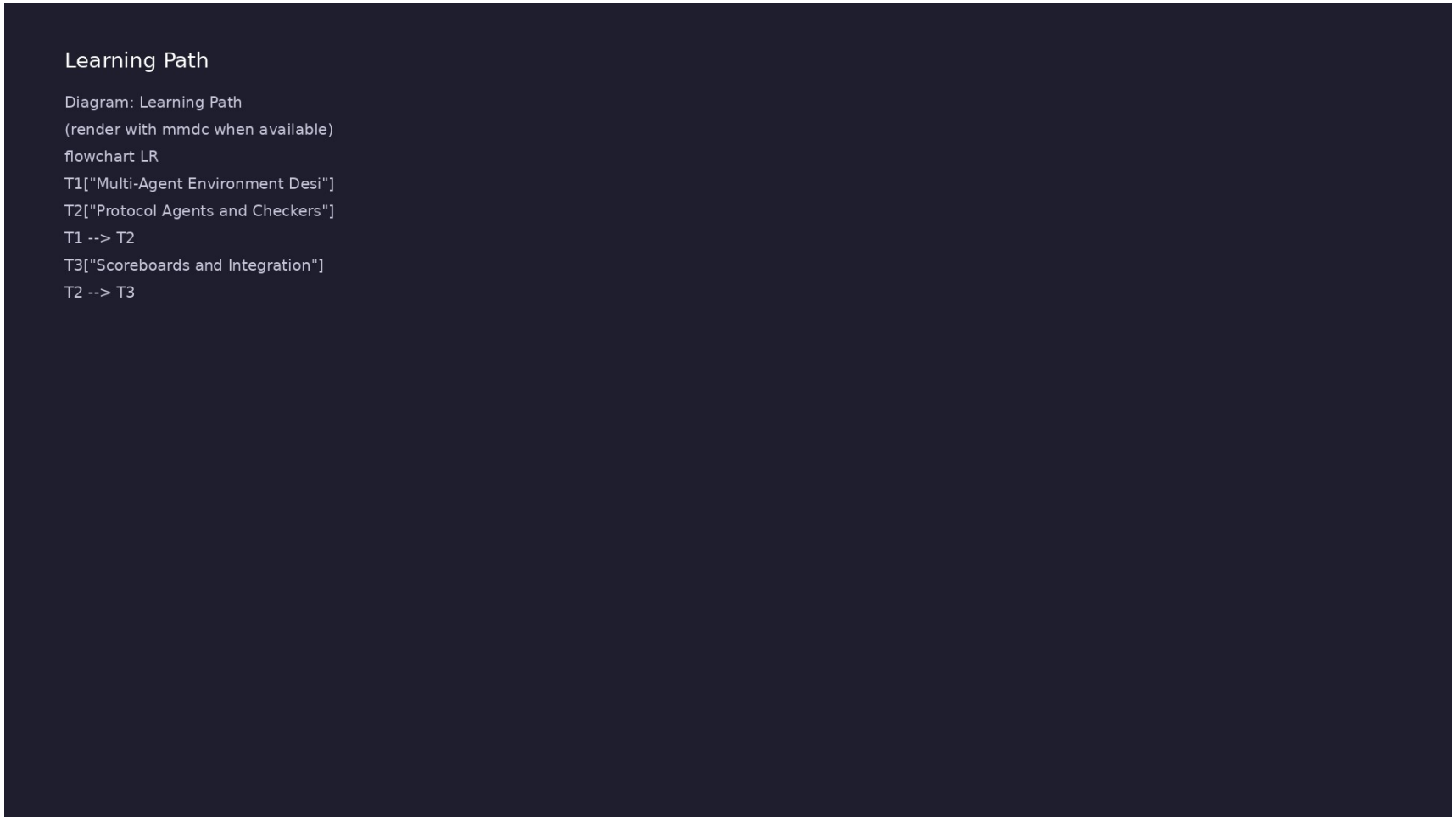
Learning objectives

- Apply your planning, environment, and regression skills to **complex multi-agent, protocol-heavy te...

Prerequisites

- See module README

Learning path



Apply your planning, environment, and regression skills to **complex multi-agent, protocol-heavy testbenches** (e.g., AXI-like, multi-chann...

Overview

- By Module 6, you have:

How to learn this module

Suggested learning path

- Re-read `module4/ENV_DESIGN.md` and `module5/REGRESSION_OPS.md`.
- Scaffold Module 6 workspace: `./scripts/module6.sh --scaffold`
- Complete `MULTI_AGENT_ARCHITECTURE.md`, `PROTOCOL_VERIFICATION_PLAN.md`, and `INTEGRATION_PLAN.md`.
- Add or refine multi-agent and protocol checker components in `common_dut/tb/`.
- Validate: `./scripts/module6.sh --check`

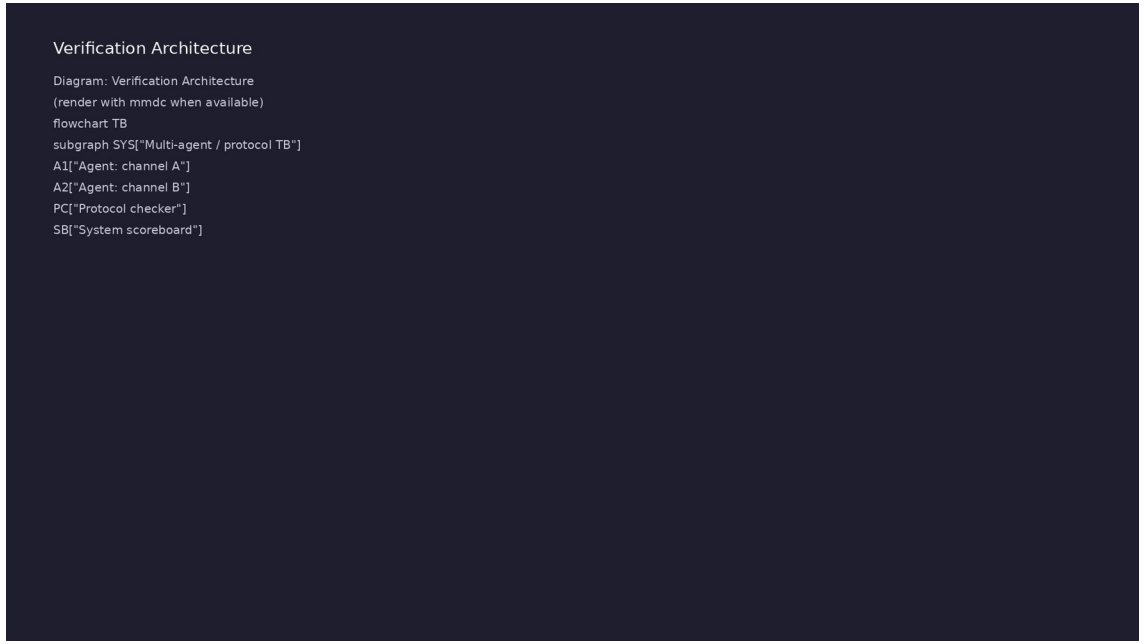
Design architecture

1. Multi-agent / multi-channel TB

- Multiple agents (per interface or channel) coordinated by env and virtual sequences.
- Layered architecture: block agents → fabric/scoreboard → subsystem scenarios (`MULTI_AGENT_ARCHITEC...

2. Protocol verification layer

- Dedicated protocol checker (SVA or monitor-based) separate from scoreboard data checks.
- `PROTOCOL_VERIFICATION_PLAN.md` lists rules, coverage, and integration with agents.



3. System integration view

- `INTEGRATION_PLAN.md` describes how block-level env connects to subsystem tests.
- Reuse agents/VIP across DUT variants via configuration and factory overrides.

Monitor observes transactions

```
class stream_monitor extends uvm_component;

  `uvm_component_utils(stream_monitor)

  virtual stream_if.slave vif;
  uvm_analysis_port #(stream_item) ap;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    ap = new("ap", this);
  endfunction

  task run_phase(uvm_phase phase);
    stream_item tr;

    if (vif == null) begin
# ...
```

Multi-agent architecture template

```
# Multi-Agent & Multi-Channel Architecture (Module 6)

> **Purpose**: Describe the architecture and coordination of multi-
agent, multi-channel UVM environments for your DUT.
> **Module**: 6 — Complex Multi-Agent & Protocol Testbenches (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from
`module6/templates/` if needed, or edit the file in `module6/`
directly.
Reference: `module6/.solutions/`.

## 1. Context

<!-- TODO: DUT/System, interfaces/channels, existing env. Why a multi-
agent design is required. -->

## 2. High-Level Environment Diagram
```

Key files to study

Open these in the repo

- `module6/MULTI_AGENT_ARCHITECTURE.md` — multi-channel env and virtual sequence coordination
- `module6/PROTOCOL_VERIFICATION_PLAN.md` — protocol rules, checkers, and coverage
- `module6/INTEGRATION_PLAN.md` — block-to-subsystem integration strategy
- `module4/tb/stream_pkg.sv` — monitor and agent patterns to extend
- `scripts/module6.sh` — multi-agent and protocol doc checks

Verification & testing methods

1. Protocol rule testing

- Each protocol rule maps to checker fires, coverpoints, and at least one directed or random test.
- Illegal sequences and corner timing documented with expected checker behavior.

2. Multi-agent scenario testing

- Virtual sequences stress cross-agent ordering, arbitration, and backpressure.
- System scoreboard validates end-to-end data and sideband signals.

3. Maintainability reviews

- Architecture review before adding agents — avoid duplicate monitors and conflicting checkers.
- `./scripts/module6.sh --check` validates multi-agent and protocol planning artifacts.

Protocol verification plan excerpt

```
# Protocol Verification Plan (Module 6)

> **Purpose**: Define how you will verify protocol correctness (e.g.,
AXI-like, custom bus) using agents, checkers, coverage, and tests.
> **Module**: 6 — Complex Multi-Agent & Protocol Testbenches (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from
`module6/templates/` if needed, or edit the file in `module6/`
directly.
Reference: `module6/.solutions/`.

## 1. Protocol Overview

<!-- TODO: Protocol name, key characteristics (channels, handshakes,
ordering, error/response semantics). How the protocol is used by the
DUT
(roles, traffic patterns). -->
```

Syllabus topics

1. Multi-Agent Environment Design

- Multi-agent and multi-channel environment design.
- Layered and reusable testbench architecture patterns.

2. Protocol Agents and Checkers

- Protocol agent and protocol checker design.

3. Scoreboards and Integration

- Integration of complex scoreboards and time-based matching.

4. Debug and Analysis

- Debugging and analyzing complex, protocol-heavy simulations.

Command reference highlights

Scaffold and validate Module 6

- ./scripts/module6.sh --scaffold

Review multi-agent architecture template

- head -40
  module6/templates/MULTI_AGENT_ARCHITECTURE.md

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
[0;36m==========================================[0m
[0;36mModule 6: Complex Multi-Agent & Protocol Testbenches[0m
[0;36m==========================================[0m

Usage: ./scripts/module6.sh [OPTIONS]

Module 6: Complex Multi-Agent & Protocol Testbenches (SV/UVM)
This script summarizes and checks the planning artifacts for Module 6.

OPTIONS:
    --multi-agent-status   Show status of multi_agent_architecture.md only
    --protocol-status      Show status of protocol_verification_plan.md only
    --integration-status   Show status of integration_plan.md only
    --checklist-status     Show status of checklist_module6.md only
    --summary              Show all statuses (default)
    --check                Media/CI self-check (structure only)
    --demo                 Print example commands from EXAMPLES.md
    --scaffold             Copy templates/*.md into module6/ if missing
    --help, -h             Show this help message

EXAMPLES:
    # Full summary
    ./scripts/module6.sh

    # Focus only on multi-agent architecture
    ./scripts/module6.sh --multi-agent-status
```

Exercise scaffold

```
./scripts/module6.sh --scaffold
```

Demo: Protocol verification plan

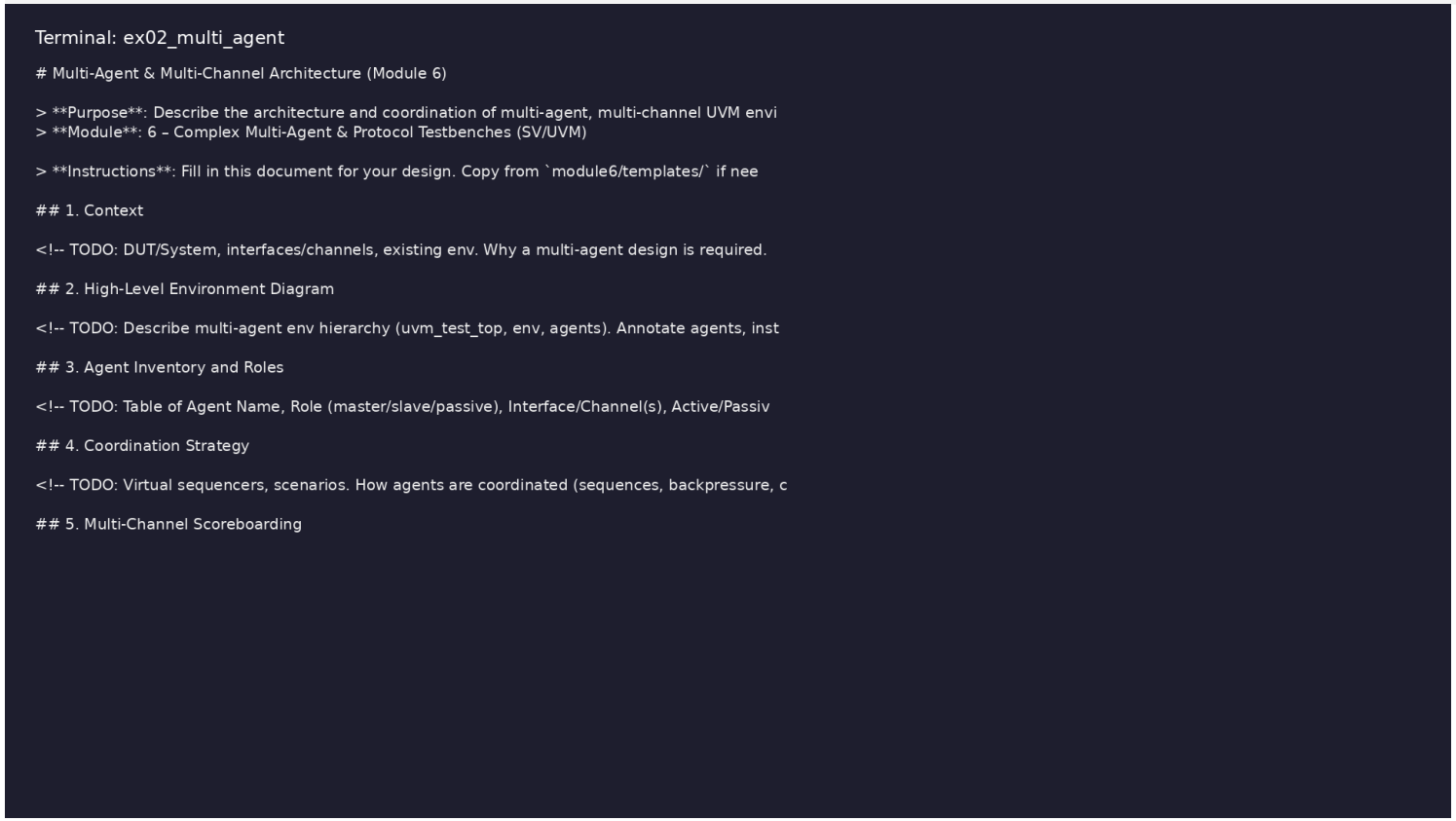
```
$ head -30 module6/templates/PROTOCOL_VERIFICATION_PLAN.md
```

Terminal: ex01_protocol_plan

```
# Protocol Verification Plan (Module 6)

> **Purpose**: Define how you will verify protocol correctness (e.g., AXI-like, custom bus) usin
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module6/templates/` if nee

## 1. Protocol Overview

<!-- TODO: Protocol name, key characteristics (channels, handshakes, ordering, error/response se

## 2. Protocol Agent Strategy

<!-- TODO: Table of Agent Name, Role (master/slave/passive), Channels Handled, Notes. Active vs

## 3. Protocol Rules and Checkers

### 3.1 Rule Inventory

<!-- TODO: Table of Rule ID, Description, Type (safety/order/timing), Related Req IDs, Notes. --

### 3.2 Checker Design

<!-- TODO: Table of Checker ID, Location, Inputs, Rules Enforced. Placement and error reporting.

## 4. Protocol Coverage

### 4.1 Functional Coverage for Protocol

<!-- TODO: Coverage IDs, purpose, placement/sampling. Link to module3/COVERAGE_DESIGN.md. -->
```

Demo: Multi-agent architecture

```
$ head -25 module6/templates/MULTI_AGENT_ARCHITECTURE.md
```



Demo: Integration plan

```
$ head -25 module6/templates/INTEGRATION_PLAN.md
```

Demo: Reference protocol plan

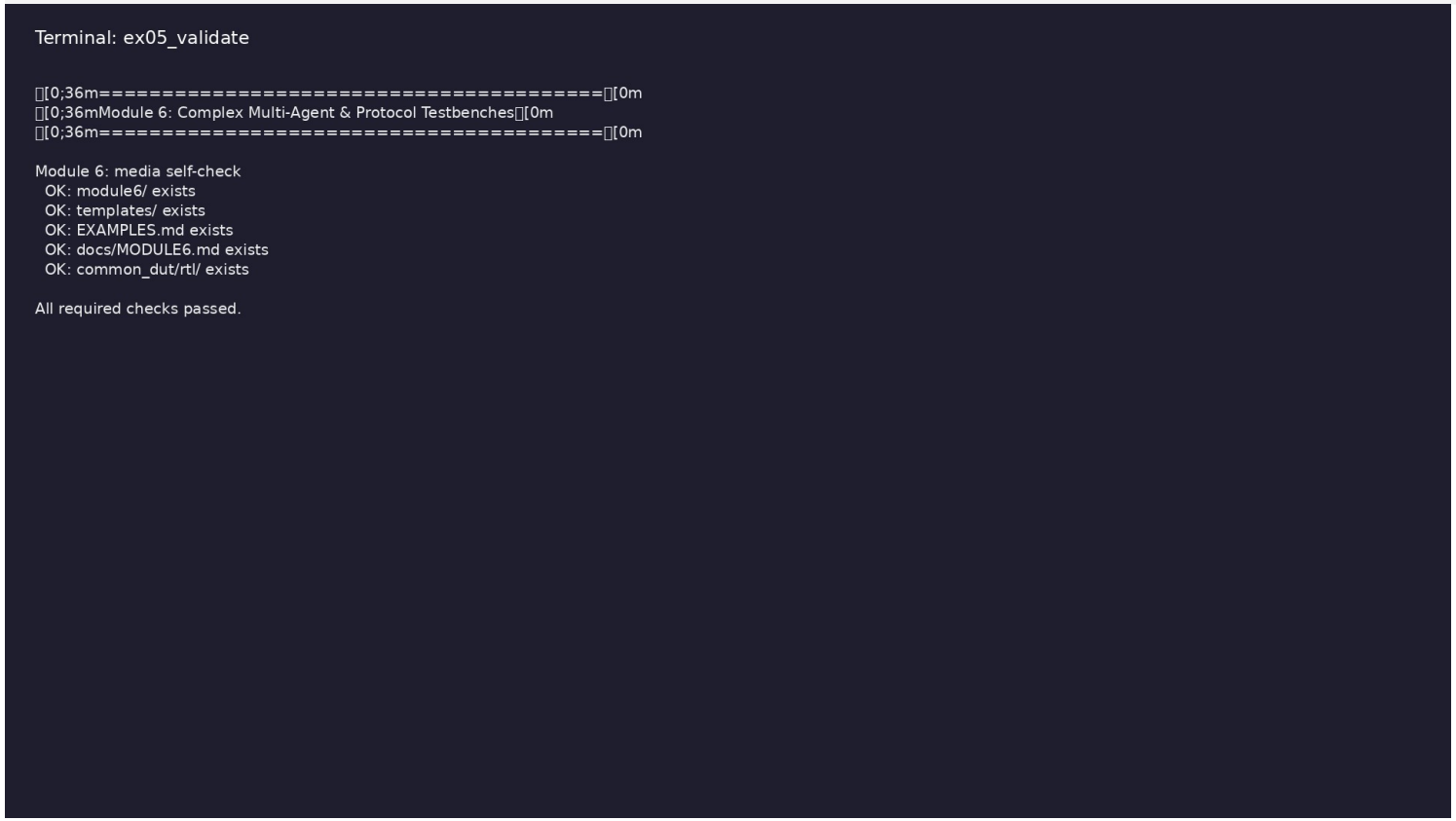
```
$ head -20 module6/.solutions/PROTOCOL_VERIFICATION_PLAN.md
```

Terminal: ex04_solutions

```
# Protocol Verification Plan (Module 6)

> **Purpose**: Define how you will verify protocol correctness (e.g., AXI-like, custom bus) usin
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)

## 1. Protocol Overview

- **Protocol Name**: Ready/Valid streaming (source and sink).
- **Key Characteristics**:
  - **Channels**: Source (s_valid, s_ready, s_data); sink (m_valid, m_ready, m_data).
  - **Handshakes**: Transfer on clock edge when valid && ready; one beat per handshake.
  - **Ordering**: FIFO order — first-in first-out.
  - **Error/response semantics**: Overflow (write when full) and underflow (read when empty) set

Summarize how the protocol is used by the DUT (roles, typical traffic patterns):

- **DUT roles**: stream_fifo is sink on source interface (accepts push) and source on sink inter
- **Traffic**: Single-beat transfers; backpressure via ready deassertion; typical patterns = sus

## 2. Protocol Agent Strategy
```

Demo: Module validation

```
$ ./scripts/module6.sh --check
```



Practice & assessment

What you should know (1/2)

- Design and document multi-agent UVM environments for protocol-based DUTs.
- Implement and integrate protocol agents and protocol checkers.
- Apply layered architecture patterns to keep complex benches maintainable.
- Analyze and debug multi-channel, protocol-level behavior in simulations.
- Multi-agent/multi-channel environment design documented in `module6/MULTI_AGENT_ARCHITECTURE.md`.
- Protocol verification strategy documented in `module6/PROTOCOL_VERIFICATION_PLAN.md`.

What you should know (2/2)

- Integration patterns and system-level scenarios documented in `module6/INTEGRATION_PLAN.md`.
- Complex bench design reviewed; action items captured in the docs.

Assessment checklist

- Multi-agent/multi-channel environment design
  documented in `module6/MULTI_AGENT_ARCHITECTURE.md`.
- Protocol verification strategy documented in
  `module6/PROTOCOL_VERIFICATION_PLAN.md`.
- Integration patterns and system-level scenarios
  documented in `module6/INTEGRATION_PLAN.md`.
- Complex bench design reviewed; action items captured
  in the docs.

Summary & next steps

- Apply your planning, environment, and regression skills to **complex multi-agent, protocol-heavy te…
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: Next module in course